

GUIA DO REPOSITÓRIO

educado-tests

Ambiente LaTeX colaborativo para os relatórios da disciplina FGA0314, Testes de Software. Como o repositório é organizado, como compilar, como escrever em equipe sem conflito e como entregar.

Identificação

REPOSITÓRIO	github.com/ErasmusEgalitarian/educado-tests
DISCIPLINA	FGA0314 - Testes de Software, FCTE/UnB
PROJETO ANALISADO	Educado (OSS)
PÚBLICO	Equipe de testes
PRÉ-REQUISITO	Nenhum: o CI compila pra quem não instalar TeX Live

Este repositório é o ambiente onde a equipe escreve os relatórios da disciplina FGA0314 (Testes de Software) sobre o projeto OSS Educado. Ele existe pra resolver um problema específico: várias pessoas escrevendo o mesmo documento LaTeX ao mesmo tempo, sem pisar no arquivo uma da outra e sem ninguém precisar instalar TeX Live na máquina.

Este guia segue a ordem em que você vai precisar das coisas: clonar, configurar, escrever, commitar, abrir PR e entregar.

0 1

A ideia em uma página

O template que o professor distribui é um arquivo único de mil linhas. Três pessoas editando esse arquivo ao mesmo tempo significa conflito de merge em toda entrega, e conflito em LaTeX é especialmente ruim porque o Git não entende que uma seção é independente da outra.

A solução tem duas camadas. Primeiro, **o documento foi quebrado em um arquivo por seção**: você edita a sua seção e ninguém mais toca nela. Segundo, **cada relatório é uma pasta isolada**, então o Relatório 2 não interfere no Relatório 1, mesmo no mesmo repositório.

O PDF gerado é idêntico ao do template original do professor. A modularização é interna, não muda o resultado impresso.

0 2

Do clone ao primeiro PDF

Passo 1: clonar

```
git clone https://github.com/ErasmusEgalitarian/educado-tests.git
cd educado-tests
```

Se você usa o GitHub CLI, `gh repo clone ErasmusEgalitarian/educado-tests` faz o mesmo.

Você precisa estar na organização ErasmusEgalitarian com acesso de escrita pra conseguir enviar branches. Se o `git push` falhar com erro de permissão, é isso: peça acesso antes de escrever.

Passo 2: escolher como compilar

São três caminhos. Escolha um, nenhum é obrigatório, e dá pra trocar depois.

Caminho A, não instalar nada. Você escreve, abre um Pull Request, e o GitHub Actions compila. Vá em **Actions**, escolha a execução do seu PR, e baixe o PDF em **Artifacts**. É o caminho recomendado pra maioria da equipe.

Caminho B, devcontainer. O repositório traz um `.devcontainer/` com TeX Live completo e o LaTeX Workshop configurado. No VS Code, abra a pasta e aceite "Reopen in Container". Pelo navegador, use **Code**, depois **Codespaces**, depois **Create codespace**. Salvou o arquivo, recompila sozinho e mostra o PDF ao lado.

Caminho C, local. Precisa de TeX Live com `latexmk`. No Ubuntu ou Debian:

```
sudo apt install texlive-full latexmk
```

O pacote é grande, alguns gigabytes. Se você só quer ver o PDF de vez em quando, o Caminho A resolve sem isso.

Passo 3: compilar

```
./build.sh                # lista os relatórios do repo
./build.sh 01-estrategia-de-testes  # compila um
./build.sh all             # compila todos
./build.sh 01-estrategia-de-testes watch  # recompila a cada save
./build.sh clean           # limpa os intermediários
```

O PDF sai em `build/`, com o nome da entrega. A pasta `build/` não é versionada.

0 3

Como o repositório é organizado

```
comum/
  preambulo.tex          formatação de TODOS os relatórios
  logo_unb.jpg
relatorios/
  01-estrategia-de-testes/
    main.tex             espinha: preâmbulo + ordem das seções
    entrega.txt          nome do PDF final, o que vai pro Moodle
    secoes/              o conteúdo, um arquivo por seção
    figuras/             imagens e evidências deste relatório
build/                   saída da compilação, não versionado
docs/                   este guia
scripts/novo-relatorio.sh cria a pasta do próximo relatório
```

As seções do Relatório 1 estão divididas assim:

ARQUIVO	SEÇÃO
<code>secoes/00-capa.tex</code>	Capa: equipe, integrantes, matrículas
<code>secoes/01-identificacao.tex</code>	Identificação do projeto, repositório, commit
<code>secoes/02-introducao.tex</code>	1. Introdução
<code>secoes/03-visao-geral.tex</code>	2. Visão Geral do Projeto
<code>secoes/04-estrategia-de-testes.tex</code>	3. Estratégia de Testes
<code>secoes/05-execucao-da-suite.tex</code>	4. Execução da Suíte de Testes
<code>secoes/06-avaliacao-critica.tex</code>	5. Avaliação Crítica
<code>secoes/07-conclusao.tex</code>	6. Conclusão
<code>secoes/08-referencias.tex</code>	Referências
<code>secoes/09-appendice.tex</code>	Apêndice: evidências complementares

Arquivos que são de todo mundo

O `main.tex` de cada relatório contém só o preâmbulo e a ordem dos `\input`. Se você editar, conflita com todo mundo daquele relatório.

O `comum/preambulo.tex` define pacotes e formatação de **todos** os relatórios. Mudar ali muda o layout dos outros também, inclusive de relatório já entregue. Avise no grupo antes.

Figura vai em `figuras/` do próprio relatório, nunca em `comum/`.

0 4

Contribuindo: o ciclo completo

A regra que evita quase toda a dor: **uma pessoa por seção, uma branch por seção, um PR por seção.**

```
git checkout main && git pull
git checkout -b docs/rel01-estrategia-de-testes
# escreve a sua seção
./build.sh 01-estrategia-de-testes
git add relatorios/01-estrategia-de-testes/secoes/04-estrategia-de-testes.tex
git commit -m "docs(rel01): descrever niveis de teste com evidencia"
git push -u origin docs/rel01-estrategia-de-testes
gh pr create --fill
```

Trabalhando em relatórios diferentes, vocês não se cruzam: pastas separadas, zero conflito.

Nome de branch

O formato é `tipo/relNN-assunto`, tudo minúsculo e com hífen.

SITUAÇÃO	BRANCH
Escrever uma seção	<code>docs/rel01-estrategia-de-testes</code>
Corrigir erro no texto	<code>fix/rel01-tabela-cobertura</code>
Começar um relatório novo	<code>feat/rel02-plano-de-testes</code>
Mexer em CI ou tooling	<code>ci/matrix-de-relatorios</code>

Antes de começar, dê `git pull` na `main`. Branch criada a partir de uma `main` velha gera conflito que não precisava existir.

0 5

Padrão de commit

O repositório usa **Conventional Commits**. O formato é:

```
tipo(escopo): descricao no imperativo
```

```
Corpo opcional explicando o porque, nao o que.
```

Tipos usados aqui

TIPO	QUANDO
<code>docs</code>	Conteúdo do relatório: escrever, reescrever, completar seção
<code>fix</code>	Corrigir erro no texto, número errado, referência quebrada
<code>feat</code>	Estrutura nova: relatório novo, seção nova, comando novo
<code>style</code>	Só formatação LaTeX, sem mudar o que o texto diz
<code>ci</code>	Workflows do GitHub Actions
<code>chore</code>	Manutenção geral: gitignore, devcontainer, dependência
<code>refactor</code>	Reorganizar arquivos sem mudar o PDF resultante

Escopo

O escopo é o relatório: `rel01` , `rel02` . Para mudanças que não pertencem a um relatório específico (CI, tooling, preâmbulo comum), omita o escopo ou use `comum` .

Regras da descrição

Verbo no imperativo: "adicionar", não "adicionado" nem "adicionando". Minúscula na primeira letra, sem ponto final, no máximo 72 caracteres. Descreva o que o commit faz, não em qual arquivo mexeu.

Bons exemplos:

```
docs(rel01): descrever niveis de teste com evidencia do repo
fix(rel01): corrigir cobertura de branches na tabela 4
feat(rel02): criar estrutura do plano de testes
ci: compilar todos os relatorios em matrix
```

Exemplos ruins e o motivo:

COMMIT	PROBLEMA
<code>update</code>	Não diz nada
<code>mexi no arquivo 04</code>	Descreve o arquivo, não a mudança
<code>docs(rel01): Adicionadas as evidencias.</code>	Particípio, maiúscula e ponto final
<code>wip</code>	Não vai pra <code>main</code> ; se precisar salvar trabalho, deixe na sua branch

O corpo

Use o corpo quando o "porquê" não for óbvio. É onde se explica uma decisão, não o que o diff já mostra. Se o commit muda um número ou conclusão do relatório, o corpo é o lugar de dizer de onde veio o dado.

0 6

Regras de Pull Request

Antes de abrir

O CI precisa compilar verde. Se ficar vermelho no passo de compilar, o LaTeX quebrou: abra o log da execução, ele aponta o arquivo e a linha exata.

O PR deve cobrir **uma seção**. PR que toca cinco seções não é revisável e trava o trabalho dos outros.

O que o PR precisa ter

O título segue o mesmo padrão do commit: `docs(rel01): descrever níveis de teste com evidencia`.

A descrição diz o que entrou e o que ficou faltando, se ficou. O repositório tem um template de PR que já carrega o checklist.

Marque **um revisor**. Revisão de colega é obrigatória: é o filtro que pega afirmação sem fonte antes do professor pegar.

O checklist

- O CI compilou verde
- Toda afirmação sobre o projeto tem fonte apontada: arquivo, issue, PR, workflow
- Evidência, avaliação e recomendação estão separadas, não misturadas
- Placeholders do template preenchidos ou removidos nas partes que toquei
- Não mexi no `main.tex` do relatório nem em `comum/preambolo.tex`, ou avisei no grupo

O que barra o merge

CI vermelho. Ausência de revisor. Afirmação sobre o projeto sem fonte apontada. Mudança em `comum/preambolo.tex` sem aviso no grupo, porque afeta relatório já entregue.

Merge

Use **squash merge**: cada PR vira um commit na `main`, e o histórico fica legível. Apague a branch depois do merge, o GitHub oferece o botão.

Não faça **force push** numa branch que já está em revisão: quebra os comentários que o revisor deixou.

0 7

As regras de escrita que valem nota

O professor é explícito sobre isso no template. Três coisas precisam estar visivelmente separadas ao longo do relatório.

Evidência encontrada é o que foi observado no projeto, sempre com a fonte apontada: arquivo, diretório, issue, pull request, workflow ou trecho de código.

Avaliação da equipe é a interpretação de vocês sobre a prática identificada, usando os conceitos da disciplina.

Recomendação é a melhoria proposta. Nunca escreva recomendação como se o projeto já fizesse aquilo.

Afirmação sem fonte é o erro mais caro do relatório. Escrever "o projeto possui testes unitários" sem apontar onde isso foi visto não conta como evidência.

LaTeX na prática

Escreva **uma linha por frase**. Facilita o diff e o review, e não muda nada no PDF: o LaTeX junta as linhas no mesmo parágrafo. É linha em branco que separa parágrafo.

Para inserir uma figura, coloque o arquivo em `figuras/` do seu relatório e referencie:

```
\begin{figure}[H]
  \centering
  \includegraphics[width=0.8\textwidth]{figuras/cobertura-jest.png}
  \caption{Relatório de cobertura gerado pelo Jest.}
  \label{fig:cobertura}
\end{figure}
```

No texto, cite com `Figura \ref{fig:cobertura}`. O mesmo vale pra tabelas, com `Tabela \ref{tab:...}`.

Log de execução ou trecho de código vai em `lstlisting`:

```
\begin{lstlisting}
npm test -- --coverage
\end{lstlisting}
```

Os caracteres `& % $ # _ { }` quebram a compilação se usados direto: precisam de barra invertida, como `\&`, `\%` e `\_`. Em URL, use `\url{...}` que resolve sozinho.

Começar um relatório novo

Quando a disciplina pedir o próximo relatório:

```
./scripts/novo-relatorio.sh 02-plano-de-testes "Plano de Testes" FGA0314_Relatorio_2
```

Os três argumentos são: nome da pasta, título que aparece na capa, e nome do PDF final. Isso cria a pasta já ligada ao preâmbulo comum, com capa e uma seção inicial.

O CI descobre a pasta sozinho na próxima execução: **nenhum workflow precisa ser editado**.

Quando o professor mandar o template do relatório novo, quebre o `main.tex` dele em `secoes/` do mesmo jeito que o Relatório 01 está. É isso que permite escrever em paralelo.

Gerar uma nova versão do documento

A entrega é uma **tag**. O CI compila aquele relatório e cria uma Release do GitHub com o PDF anexado.

Primeira entrega

```
git checkout main && git pull
git tag entrega-01-estrategia-de-testes
git push origin entrega-01-estrategia-de-testes
```

O trecho depois de `entrega-` é o nome da pasta em `relatorios/`.

Reentrega ou versão nova

Se o relatório precisar ser reentregue, com correção depois de feedback, acrescente o sufixo de versão:

```
git tag entrega-01-estrategia-de-testes-v2
git push origin entrega-01-estrategia-de-testes-v2
```

O CI entende o `-v2` como versão, encontra a mesma pasta, e cria uma Release separada identificada como v2. As versões anteriores continuam disponíveis, então dá pra recuperar exatamente o PDF que foi entregue antes.

TAG	PASTA COMPILADA	RELEASE
<code>entrega-01-estrategia-de-testes</code>	<code>01-estrategia-de-testes</code>	v1
<code>entrega-01-estrategia-de-testes-v2</code>	<code>01-estrategia-de-testes</code>	v2
<code>entrega-02-plano-de-testes-v3</code>	<code>02-plano-de-testes</code>	v3

Onde pegar o PDF

Na aba **Releases** do repositório. O arquivo já sai com o nome definido no `entrega.txt` do relatório, que é o nome esperado no Moodle.

Se a tag apontar pra uma pasta que não existe, o CI falha e lista as pastas disponíveis, em vez de gerar um PDF errado silenciosamente.

Antes de marcar a tag

Confira que a `main` está com tudo que precisa entrar, que o CI da `main` está verde, e que os placeholders do template sumiram. A tag congela um ponto do histórico: se estiver errado, a correção é uma tag nova, não uma edição da anterior.

Quando algo quebra

SINTOMA	CAUSA PROVÁVEL	O QUE FAZER
CI vermelho no passo de compilar	Erro de LaTeX na sua seção	Abrir o log da execução: aponta arquivo e linha
<code>git push</code> recusado	Sem acesso de escrita na organização	Pedir acesso à ErasmusEgalitarian
Figura não aparece	Caminho errado, ou arquivo não commitado	Conferir se o arquivo foi commitado junto
<code>Undefined control sequence</code>	Comando com erro de digitação, ou pacote faltando	Conferir a grafia; pacote novo entra no <code>comum/preambulo.tex</code>
Caractere quebra a build	<code>& % \$ # _</code> sem escapar	Colocar barra invertida antes
Conflito de merge no PR	Duas pessoas na mesma seção	Combinar quem fica com o arquivo; é uma pessoa por seção
Release não apareceu	Tag com nome de pasta errado	Ver o log do workflow "Publicar entrega": ele lista as pastas válidas